Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

Лабораторная работа 3

Формы и локальное состояние приложения

Продолжите проект второй лабораторной. Пользователь сможет создавать, редактировать и удалять записи, а список и карточка будут сразу показывать изменения. Страница создания перестанет быть заглушкой. Для поиска нужных записей вы добавите один локальный фильтр.

Цель работы

Связать формы и страницы с общим состоянием React, обрабатывать события и проверять ввод. Научиться изменять массив объектов без мутаций и отделять несохранённый черновик формы от данных приложения.

Обязательный результат

Локальный CRUD основной сущности: создание, просмотр, изменение и удаление. Одна переиспользуемая форма для создания и редактирования, отмена без сохранения, обязательные поля и предметная проверка, один фильтр и понятные пустые состояния. Результат находится в том же Git-репозитории и проходит сборку с проверкой типов.

Данные пока живут только в памяти открытого приложения. После обновления страницы возвращаются демонстрационные записи ЛР 2. Добавьте видимое пояснение об этом в общий layout. Постоянное хранение появится вместе с Supabase; localStorage не требуется. Внутренние переходы между страницами должны сохранять текущие изменения.

Теоретическая справка

Общее состояние и черновик формы

useState хранит значение между перерисовками компонента и предоставляет функцию обновления. Изменение обычной переменной не сообщает React, что интерфейс нужно обновить. Общий список должен находиться в компоненте,

который остаётся смонтированным при переходах между страницами: в нашем проекте это App.

Страницы получают список и функции изменения через props. Для этого объёма достаточно поднять состояние в App; Context и отдельная библиотека состояния не нужны. Массив из data/tasks.ts используется только при инициализации. Если карточка продолжит читать его напрямую, она не увидит новую запись.

Форма хранит отдельный черновик. Ввод меняет его, а кнопка сохранения переносит проверенные значения в общий список. Отмена просто закрывает форму. При редактировании черновик начинается с копии полей записи, поэтому несохранённый текст не меняет карточку.

События и неизменяемые обновления

Управляемое поле получает value из состояния и обновляет его через onChange. Отправку формы обрабатывает onSubmit; preventDefault отменяет стандартную перезагрузку страницы. Кнопки внутри формы имеют явный type: submit для сохранения и button для отмены.

Для нового состояния создают новый массив: добавление через spread, изменение через map, удаление через filter. Запись определяется стабильным id. Функциональная форма setTasks(current => ...) использует актуальное предыдущее значение; функция обновления должна оставаться чистой.

Практическая часть

Шаг 1 Подготовьте существующий проект

В корне проверьте git status и сохраните завершённую ЛР 2. Из frontend выполните npm ci и npm run build. Дополнительные зависимости для этой работы не нужны. Сохраните BrowserRouter в main.tsx, общий layout и обработку неизвестных адресов.

Примеры продолжают Task Tracker из ЛР 2. Заменяйте Task, tasks и названия маршрутов сущностью своей темы. Для всех вариантов сохраняется один

сценарий: форма создаёт черновик, общий компонент хранит список, карточка ищет запись по URL.

Шаг 2 Опишите данные формы

В frontend/src/types/task.ts сохраните типы Task и TaskStatus из второй работы и добавьте TaskDraft. id создаётся приложением и не редактируется пользователем:

```
export type TaskDraft = Omit<Task, 'id'>
```

TaskDraft включает title, description, status, dueDate, priority и tag. Для пустой формы задайте значения по умолчанию: текстовые поля пустые, статус todo, приоритет medium. Если у вашей сущности есть число или checkbox, учтите особенности ввода из шага 5.

Шаг 3 Перенесите список в App

В frontend/src/App.tsx импортируйте useState, Task и TaskDraft. Демонстрационный массив импортируйте под именем initialTasks. Внутри App, перед return с маршрутами, создайте состояние и три функции. Сохраните уже существующие импорты страниц и маршрутизатора.

```
import { useState } from 'react'
import type { Task, TaskDraft } from './types/task'
import { tasks as initialTasks } from './data/tasks'

// Внутри функции App перед return:
const [tasks, setTasks] = useState<Task[]>(() =>
  initialTasks.map((task) => ({ ...task }))
)

function createTask(draft: TaskDraft): string {
  const id = crypto.randomUUID()
  setTasks((current) => [...current, { ...draft, id }])
  return id
}

function updateTask(id: string, draft: TaskDraft): void {
  setTasks((current) => current.map((task) =>
    task.id === id ? { ...draft, id: task.id } : task
  ))
}

function deleteTask(id: string): void {
  setTasks((current) => current.filter((task) => task.id !== id))
}
```

Генерируйте id один раз при создании, до вызова функции обновления состояния. Внутри setTasks не показывайте confirm и не вызывайте навигацию. React может повторно вызывать чистые функции обновления в режиме разработки. crypto.randomUUID доступен в безопасном контексте, включая localhost; в курсе приложение запускается через Vite.

Удалите прямой импорт data/tasks.ts из TasksPage и TaskDetailsPage. Теперь эти страницы принимают tasks через props. TaskCard по-прежнему получает одну запись. Начальные данные больше нигде не должны подменять изменяемый список.

Шаг 4 Задайте контракты страниц и маршруты

Используйте следующие props; типы можно разместить в файлах соответствующих страниц. NewTaskPage создаёт запись, EditTaskPage изменяет найденную запись, а TaskDetailsPage получает обработчик удаления.

```
type TasksPageProps = { tasks: Task[] }
type NewTaskPageProps = {
  onCreate: (draft: TaskDraft) => string
}
type EditTaskPageProps = {
  tasks: Task[]
  onUpdate: (id: string, draft: TaskDraft) => void
}
type TaskDetailsPageProps = {
  tasks: Task[]
  onDelete: (id: string) => void
}
```

Добавьте EditTaskPage в frontend/src/pages/EditTaskPage.tsx. В существующем Route с AppLayout замените маршруты списка, создания и карточки и добавьте редактирование. Главный редирект и маршрут * из ЛР 2 сохраните. Ниже — фрагмент внутри Routes, а не отдельное приложение:

```
<Route path="tasks" element={<TasksPage tasks={tasks} />} />
<Route path="tasks/new" element={
  <NewTaskPage onCreate={createTask} />
} />
<Route path="tasks/:id" element={
  <TaskDetailsPage tasks={tasks} onDelete={deleteTask} />
} />
<Route path="tasks/:id/edit" element={
  <EditTaskPage tasks={tasks} onUpdate={updateTask} />
} />
```

App остаётся владельцем состояния при смене маршрута. Не переносите useState(tasks) в каждую страницу: это создаст независимые списки. Все хуки вызывайте на верхнем уровне компонента, до условных возвратов.

Шаг 5 Создайте переиспользуемую форму

Создайте frontend/src/components/TaskForm.tsx. Компонент получает начальные значения и два обработчика. Ниже показана логика формы. Следующий фрагмент с JSX нужно поместить в той же функции после handleSubmit, завершив компонент закрывающей фигурной скобкой.

```
import { useState } from 'react'
import type { FormEvent } from 'react'
import type { TaskDraft } from '../types/task'

type TaskFormProps = {
  initialValues: TaskDraft
  onSave: (draft: TaskDraft) => void
  onCancel: () => void
}

export function TaskForm(props: TaskFormProps) {
  const [draft, setDraft] = useState(() => ({ ...props.initialValues }))
  const [error, setError] = useState('')

  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    const title = draft.title.trim()
    if (title.length < 3 || title.length > 100) {
      setError('Название должно содержать от 3 до 100 символов.')
      return
    }
    if (!draft.dueDate) {
      setError('Укажите срок задачи.')
      return
    }
    setError('')
    props.onSave({ ...draft, title })
  }
```

Добавьте JSX формы с текстом, датой и статусом. noValidate позволяет показать единое сообщение вашей проверки вместо браузерной подсказки. required остаётся признаком обязательного поля. Допишите поля описания, приоритета и тега по тому же принципу; все содержательные поля должны быть доступны при редактировании.

```
  return (
    <form onSubmit={handleSubmit} noValidate>
      <label htmlFor="task-title">Название</label>
      <input id="task-title" value={draft.title} required
        onChange={(e) => setDraft({ ...draft, title: e.target.value })}
      />
      <label htmlFor="task-date">Срок</label>
      <input id="task-date" type="date" value={draft.dueDate} required
        onChange={(e) => setDraft({ ...draft, dueDate: e.target.value })}
      />
      <label htmlFor="task-status">Статус</label>
      <select id="task-status" value={draft.status} onChange={(e) => {
        const status = e.target.value
        if (status === 'todo' || status === 'in_progress' || status === 'done') {
          setDraft({ ...draft, status })
        }
      }}>
        <option value="todo">Запланировано</option>
        <option value="in_progress">В работе</option>
        <option value="done">Готово</option>
      </select>
      {error && <p role="alert">{error}</p>}
      <button type="submit">Сохранить</button>
      <button type="button" onClick={props.onCancel}>Отмена</button>
    </form>
  )
}
```

Для description используйте textarea, для priority — select со значениями low, medium и high, для tag — текстовое поле. Свяжите label и поле через htmlFor/id. Строковые поля начинаются с пустой строки, а не undefined; значение checkbox берётся из checked.

Если есть сумма, оценка или другое число, event.target.value всё равно возвращает строку. Для поля с возможным пустым вводом храните строку в черновике формы, а при сохранении отдельно проверьте пустоту, преобразуйте в число и убедитесь в Number.isFinite и допустимом диапазоне. Не превращайте пустую строку в ноль без явного решения.

Шаг 6 Добавьте предметную проверку

Кроме обязательных полей реализуйте минимум одно ограничение своей темы и покажите понятное сообщение без потери заполненной формы. Проверка выполняется до изменения общего списка. Для Task Tracker дополните пример

правилом: завершённая задача должна иметь непустое описание результата. Прошедший срок сам по себе допустим — приложение должно уметь хранить просроченные задачи.

Остальные варианты: issue — описание не короче 10 символов; расход или доход — положительная сумма; фильм/игра — оценка от 1 до 10 либо не задана; мероприятие — окончание позже начала; привычка — дата остановки не раньше даты начала; учебное задание — сложность в выбранном диапазоне; книга — оценка от 1 до 5 либо не задана; CRM — корректный формат заполненного email. Дополнительные ограничения согласуйте со смыслом приложения.

Типы помогают во время разработки, а проверка формы — при вводе. Не отключайте strict и не используйте any, чтобы скрыть несовпадение типов. Сложная библиотека валидации для этой работы не требуется.

Шаг 7 Подключите создание записи

В NewTaskPage замените заглушку на TaskForm. Импортируйте TaskForm, TaskDraft и useNavigate. Начальные значения можно объявить константой вне компонента. Ниже показаны логика и разметка страницы; тип NewTaskPageProps определён в шаге 4.

```
const emptyTask: TaskDraft = {
  title: '', description: '', status: 'todo',
  dueDate: '', priority: 'medium', tag: '',
}

export function NewTaskPage({ onCreate }: NewTaskPageProps) {
  const navigate = useNavigate()
  function handleSave(draft: TaskDraft) {
    const id = onCreate(draft)
    navigate(`/tasks/${id}`)
  }
  return (
    <section>
      <h1>Создание задачи</h1>
      <TaskForm initialValues={emptyTask} onSave={handleSave}
        onCancel={() => navigate('/tasks')} />
    </section>
  )
}
```

После сохранения открывается карточка нового id, а список содержит новую запись. Отмена возвращает к списку и ничего не добавляет. Не создавайте запись при каждом изменении поля и не присваивайте новый id во время рендера.

Шаг 8 Подключите редактирование

На странице подробностей добавьте Link на /tasks/:id/edit. EditTaskPage получает общий список, находит запись через useParams и передаёт копию её полей в тот же TaskForm. При неизвестном id покажите сообщение и ссылку к списку, как в ЛР 2. Импортируйте необходимые типы, TaskForm, Link, useParams и useNavigate.

```
export function EditTaskPage({ tasks, onUpdate }: EditTaskPageProps) {
  const { id } = useParams()
  const navigate = useNavigate()
  const task = tasks.find((item) => item.id === id)
  if (!task) {
    return <section><h1>Задача не найдена</h1>
      <Link to="/tasks">К списку</Link></section>
  }
  const initialValues: TaskDraft = {
    title: task.title, description: task.description,
    status: task.status, dueDate: task.dueDate,
    priority: task.priority, tag: task.tag,
  }
  return (
    <section>
      <h1>Редактирование задачи</h1>
      <TaskForm key={task.id} initialValues={initialValues}
        onSave={(draft) => {
          onUpdate(task.id, draft)
          navigate(`/tasks/${task.id}`)
        }}
        onCancel={() => navigate(`/tasks/${task.id}`)} />
    </section>
  )
}
```

key={task.id} сбрасывает черновик при переходе к другой редактируемой записи. useState использует initialValues при создании формы, а не при каждом изменении props. Не синхронизируйте черновик с общим списком на каждое нажатие клавиши: это ломает отмену. id при сохранении остаётся прежним.

Шаг 9 Добавьте удаление с подтверждением

В TaskDetailsPage используйте tasks и onDelete из props и добавьте useNavigate до условного возврата для отсутствующей записи. Кнопку удаления показывайте только для найденной записи. В этом этапе допустимо обычное окно window.confirm:

```
<button type="button" onClick={() => {
  if (window.confirm(`Удалить задачу «${task.title}»?`)) {
    onDelete(task.id)
    navigate('/tasks', { replace: true })
  }
}}>
  Удалить
</button>
```

Отмена в окне оставляет запись на месте. После подтверждения пользователь возвращается к списку. Если затем открыть адрес удалённой записи в том же сеансе, появится сообщение «Задача не найдена». Удаление определяется id, поэтому совпадающие названия не должны приводить к удалению нескольких записей.

Шаг 10 Добавьте локальный фильтр

В TasksPage оставьте только локальное состояние выбранного фильтра. Сам список приходит из App. Ниже приведён фильтр по статусу; импортируйте useState и TaskStatus. Тип фильтра включает отдельное значение all.

```
const [status, setStatus] = useState<TaskStatus | 'all'>('all')
const visibleTasks = tasks.filter((task) =>
  status === 'all' || task.status === status
)
```

Добавьте select с вариантами «Все», «Запланировано», «В работе» и «Готово». Проверяйте значение из события так же, как в TaskForm, затем вызывайте setStatus. В существующем map используйте visibleTasks вместо tasks. Добавьте кнопку «Сбросить фильтр», устанавливающую all.

Различайте два состояния: tasks.length === 0 означает, что записей вообще нет; visibleTasks.length === 0 при непустом tasks означает отсутствие совпадений.

Фильтр не удаляет записи и не сохраняет отфильтрованный массив в общий список. После изменения статуса запись должна появляться или исчезать в текущем отборе.

Для другой темы используйте один естественный фильтр: статус чтения/просмотра/прохождения, тип операции, курс, категория события или активность привычки. Сохранение фильтра после ухода со страницы необязательно. Поиск и дополнительные параметры будут развиваться позже.

Шаг 11 Проверьте сценарии и сохраните работу

Запустите npm run dev из frontend. На всех страницах должно быть видно, что изменения временные. Выполните проверки в одном сеансе браузера, без обновления страницы до последнего пункта.

1. Создайте запись с корректными данными, откройте её карточку и вернитесь к списку.
2. Измените несколько полей и статус; проверьте карточку и список.
3. Внесите другие изменения и нажмите «Отмена»: сохранённая запись не меняется.
4. Попробуйте пустое и состоящее из пробелов название, отсутствие обязательного значения и нарушение предметного правила. Ошибка видна, введённые поля не исчезают.
5. Переключите фильтр и сбросьте его; проверьте отсутствие совпадений.
6. Отмените удаление, затем подтвердите его. Откройте адрес удалённой записи и неизвестный адрес редактирования.
7. Удалите все записи, увидьте пустой список и создайте новую из него.
8. Перейдите к редактированию разных записей и проверьте соответствие полей выбранному id.
9. Обновите страницу: демонстрационный набор восстановится. Для нового, не входящего в него id сообщение об отсутствии записи ожидаемо.

Проверьте отправку формы клавишей Enter, отмену без случайного submit, читаемость ошибки и подписей, работу на узком экране. При внутренней навигации не должно быть полной перезагрузки документа. В консоли не должно быть ошибок, предупреждений о неконтролируемых полях или нарушении порядка хуков.

Остановите dev, выполните npm run build и npm run preview. Повторите создание, редактирование и отмену в готовой сборке. Обновите README с доступными действиями и ограничением хранения в памяти. В docs/progress.md добавьте ЛР 3 и шаги проверки. Из корня проекта сохраните результат:

```
git status
git add .
git diff --cached --stat
git commit -m "Lab 3: add local forms and CRUD"
git push
```

Отправьте актуальные исходники и инструкции. В чистой копии репозитория выполните npm ci и npm run build из frontend. ЛР 1 и ЛР 2 остаются в истории этого же проекта.

Частые проблемы

Созданная запись не видна в карточке. Страница, вероятно, всё ещё импортирует начальный массив. Читайте tasks из props. Если записи теряются при внутренних переходах, проверьте положение общего useState и использование Link/useNavigate вместо перезагрузки.

Отмена сохраняет изменения. Форма изменяет общий объект или вызывает onSave в onChange. Храните отдельный черновик и записывайте его только после проверки submit. Все поля Task в примере плоские; для вложенных объектов нужна копия изменяемого вложенного уровня.

После изменения props поля остались от другой записи. Проверьте key формы при смене id. Не генерируйте случайный key на каждом рендере: это будет сбрасывать ввод и фокус.

Поле не даёт печатать или React сообщает о смене controlled/uncontrolled. У value должен быть соответствующий onChange и корректное начальное значение. Если ошибка «Rendered more hooks», перенесите хуки выше условных возвратов.

Фильтр потерял часть данных. Не вызывайте setTasks с отфильтрованным набором. Отбор вычисляется отдельно из полного массива. Если список не обновляется, проверьте отсутствие push, splice и присваиваний полям существующих объектов состояния.

Использование ИИ при выполнении работы

Можно попросить ИИ разобрать конкретный обработчик события или объяснить, почему отмена меняет исходную запись. Передавайте небольшой фрагмент состояния, формы и callback. Самостоятельно проверьте отмену, неизвестный id и все правила валидации. Вы должны уметь объяснить, где хранится общий список и в какой момент черновик становится записью.

Результаты работы и сдача

В репозитории находятся общий список в состоянии App, функции создания/изменения/удаления, тип черновика, переиспользуемая форма, страницы создания и редактирования, обновлённые список и карточка, фильтр и инструкции. В интерфейсе есть пояснение о временном хранении. Предоставьте ссылку на репозиторий и коммит ЛР 3.

На демонстрации создайте запись, измените её, отмените несохранённое изменение, покажите ошибку валидации, примените фильтр и удалите запись с подтверждением. Объясните, почему после обновления возвращаются исходные данные. Сохранение между перезагрузками не является критерием этой работы.

Критерии оценки

Максимум — 10 баллов. Обязательный результат должен работать целиком, включая отмену и предметную проверку.

Функционал — 5 баллов: создание и просмотр новой записи (1); редактирование той же формой с сохранением id (1); удаление с подтверждением (1); общий источник данных для списка и карточки (1); один локальный фильтр со сбросом (1).

Ошибки и крайние случаи — 2 балла: обязательные поля, предметное ограничение и отмена без сохранения (1); пустые состояния, отсутствующая запись, корректные поля и явное пояснение о временном хранении (1).

Непрерывность проекта — 1 балл: используются маршруты, типы, карточки и layout ЛР 2; новая работа развивает прежнее приложение, навигация и сборка сохранены.

Git и воспроизводимость — 2 балла: исходники и коммит доступны преподавателю (1); README и запись этапа актуальны, чистая копия устанавливается и собирается (1).

Вопросы для защиты

1. Почему список хранится в App, а черновик — в форме?
2. Чем props отличаются от локального состояния?
3. Почему push и изменение task.title напрямую здесь не подходят?
4. Что делает preventDefault при отправке формы?
5. Как отмена оставляет исходную запись без изменений?
6. Зачем форме key при смене редактируемого id?
7. Почему фильтр не должен заменять полный массив?
8. Что произойдёт после обновления страницы и почему?

Материалы для самостоятельной работы

React — общее состояние: https://react.dev/learn/sharing-state-between-components

React — обновление массивов: https://react.dev/learn/updating-arrays-in-state

React — управляемые поля: https://react.dev/reference/react-dom/components/input

React — сохранение и сброс состояния: https://react.dev/learn/preserving-and-resetting-state

React Router — переход из обработчика: https://reactrouter.com/api/hooks/useNavigate